Progress Report 4

Automated Product Inquiry Response System for Light Recycling Program

Student Name: Namesh Mathara Arachchi Vidanalage

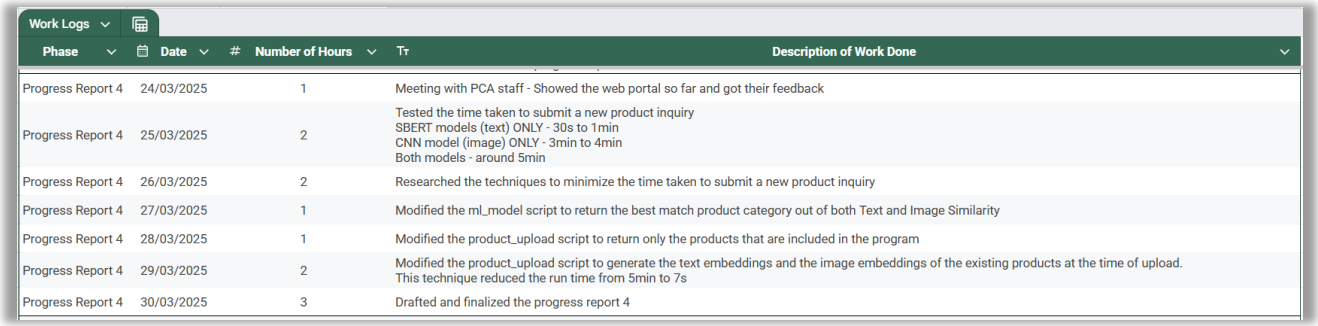
Student ID: 300359798

Course: CSIS 4495 – Applied Research Project

Section: 002

Contents

Work Logs – Summarized



Work Logs

Phase	Date	# Number of Hours	Description of Work Done
Progress Report 4	24/03/2025	1	Meeting with PCA staff - Showed the web portal so far and got their feedback
Progress Report 4	25/03/2025	2	Tested the time taken to submit a new product inquiry SBERT models (text) ONLY - 30s to 1min CNN model (image) ONLY - 3min to 4min Both models - around 5min
Progress Report 4	26/03/2025	2	Researched the techniques to minimize the time taken to submit a new product inquiry
Progress Report 4	27/03/2025	1	Modified the ml_model script to return the best match product category out of both Text and Image Similarity
Progress Report 4	28/03/2025	1	Modified the product_upload script to return only the products that are included in the program
Progress Report 4	29/03/2025	2	Modified the product_upload script to generate the text embeddings and the image embeddings of the existing products at the time of upload. This technique reduced the run time from 5min to 7s
Progress Report 4	30/03/2025	3	Drafted and finalized the progress report 4

Link to Work Logs Sheet - ***https://docs.google.com/spreadsheets/d/1GEJrsX2NxDs6eYnWCq-M8xBMALygzySMiK9KGYYx6F8/edit?pli=1&gid=0#gid=0***

Summary

1. Research & Learning

- Explored effective methods to improve performance and minimize execution time of the ml_model script:
  - **Precompute and Store Embeddings for existing products**
  - **Batch the Inference**
  - **Use Lighter Models**
  - **Use FAISS for Fast Similarity Search**
  - **Use a Background Worker**
  - **Parallelize Image Downloads & Processing**

Read more in 'Work Logs - Detailed' section

2. Model Implementation

- Updated **ml_model** script to return the best match product category based on both of Text and Image Similarity.
- Modified **product_upload** script to filter and return only the products that are included in the recycling program.
- Amended **product_upload** script to generate and store text and image embeddings for all existing products at the time of upload, rather than on each new product submission.

Read more in 'Work Logs - Detailed' section

3. Reporting & Documentation

- Drafted and finalized **Progress Report 4** summarizing the key developments.

Work Logs – Detailed

Research & Learning

Strategies to Optimize Script Performance and Reduce Runtime

Both the Text Similarity models (SBERT), and Image Similarity models (ResNet50) are computationally intensive and consume significant system resources. This is primarily because SBERT involves deep transformer-based architecture optimized for semantic understanding, while ResNet50 is a deep convolutional neural network with 50 layers, designed for high-accuracy image feature extraction. Due to their complexity, running these models - especially together - significantly slows down the ml_model script.

As a result, when a user submits a new product inquiry, they may experience a delay of up to 5 minutes before receiving a response, which is not acceptable for a real-time application.

To break it down:

- Text Similarity (**SBERT**) alone takes **30 seconds to 1 minute**
- Image Similarity (**ResNet50**) alone takes **3 to 4 minutes**
- Running **both models together** results in a total processing time of approximately **5 minutes**

To address this bottleneck, I explored various optimization strategies to reduce the runtime and improve user experience.

1. **Precompute and Store Embeddings for existing products**
2. **Batch the Inference**
3. **Use Lighter Models**
4. **Use FAISS for Fast Similarity Search**
5. **Use a Background Worker**
6. **Parallelize Image Downloads & Processing**

1. Precompute and Store Embeddings for existing products

In the current setup, **every time** a new product inquiry is submitted, the script **processes all 238 existing products** - downloading their **images** and calculating both **text** and **image embeddings** from scratch. This repetitive process is not only **resource-intensive**, but it also leads to **significant delays** in response time.

To streamline operations and enhance system **efficiency**, a smarter approach would be to **precompute** the **embeddings** - both text and image - for all existing products. These can then be stored in an **Azure Table**, **SQL database**, or a **.npy file**. By doing so, the **ml_model script** can simply retrieve the stored embeddings whenever a new inquiry comes in, **eliminating the need for recalculation**.

This optimization would drastically **improve performance**, allowing the system to deliver responses that are **faster**, **smoother**, and **more efficient**.

2. Batch the Inference

One of the most effective ways to boost performance in machine learning workflows is through a technique known as **batching inference**. Rather than processing inputs one by one, **batching** allows multiple inputs to be handled **together in a single pass** through the model. This approach is especially beneficial for deep learning architectures like **ResNet50** - commonly used for generating **image embeddings** - and **SBERT** - a powerful model for **text embeddings**.

In the current implementation, each product's **image** and **text** are processed **individually**, which means the model is repeatedly **loaded and invoked** for every single item. This repetitive process introduces **unnecessary overhead** and fails to take full advantage of available **computational resources**, especially when running on **GPUs** or **high-performance CPUs**.

By **grouping inputs** - for example, stacking multiple **preprocessed images** or collecting a batch of **product descriptions** - into a single input array, the model can compute all embeddings in one efficient pass. This not only leverages **vectorized operations** but also allows for **parallel computation**, drastically reducing the overall runtime. Instead of executing **238 separate prediction calls** for 238 products, the system can now process them all in a **single batch**, delivering results much faster.

This batching strategy becomes even more powerful when **precomputing embeddings** for existing products or managing **multiple user inquiries** in a queue. It transforms the processing pipeline into one that is both **scalable** and **resource-efficient** - a major step toward a high-performance production system.

3. Use Lighter Models

Another powerful way to enhance performance - especially in **real-time applications** or environments with **limited hardware** - is by switching to a **lighter model**. This strategy involves replacing heavy, **resource-intensive deep learning models** with smaller, faster alternatives that still deliver **reasonably accurate results**.

Take **ResNet50**, for example - a **50-layer deep convolutional neural network** that excels at **image classification** and **feature extraction** but comes at the cost of high **computational demand**. On the text side, models like **all-mpnet-base-v2** and **bert-large-nli-stsb-mean-tokens** under **SBERT** provide **top-tier embedding quality** but are considerably **slower to run** and consume much more **memory and processing power**.

In situations where **speed** and **efficiency** take priority, these heavyweight models can be replaced with **lightweight alternatives**. For image embeddings, models like **MobileNetV2** or **EfficientNetB0** offer a remarkable balance between **speed and accuracy**. On the text side, models such as **MiniLM** or **distilBERT** variants can dramatically cut down **inference time** while still performing well for most tasks.

For instance, **MobileNetV2** can generate image embeddings in a **fraction of the time** it takes **ResNet50**, making it an excellent choice for applications that demand **quick response times**. By adopting a lighter model, you not only **reduce processing time** and **lower hardware costs** but also deliver a smoother and more responsive **user experience** - particularly in cases where a small trade-off in **precision** is acceptable in favor of **speed**.

4. Use FAISS for Fast Similarity Search

To overcome the performance challenges of large-scale similarity searches, one of the most effective tools available is **FAISS** - short for **Facebook AI Similarity Search**. Developed by **Facebook**, this powerful library is specifically engineered for **fast similarity search** and **clustering of dense vectors**, making it a perfect fit for applications that rely on **embedding comparisons**.

In the current system, once **embeddings** have been generated for all **product images** and **descriptions**, the next step is to find the most **similar match** when a new inquiry is submitted. While basic methods like **brute-force cosine similarity** (e.g., using **sklearn**) may work initially, they quickly become inefficient as the dataset grows. With hundreds or even thousands of embeddings to compare, response times begin to drag.

This is where **FAISS** shines. It allows you to **index all the embeddings** into a highly optimized internal structure, enabling **rapid nearest-neighbor searches** even in large-scale datasets. What might take **seconds or minutes** with traditional methods can be accomplished in **milliseconds** using FAISS.

Better yet, FAISS offers flexibility: we can choose between **exact searches** using the **Flat index** or opt for **approximate searches** using more advanced strategies like **IVF** (Inverted File) or **HNSW** (Hierarchical Navigable Small World), depending on whether **speed** or **accuracy** is our top priority.

By integrating FAISS into your system, we can **pre-index** all the precomputed **text** and **image embeddings** and allow the **ml_model script** to retrieve the **closest match** with **minimal latency**. This significantly boosts both **performance** and **scalability**, making it ideal for **production environments** where users expect **instant feedback**.

5. Use a Background Worker

Incorporating a **background worker** into the system is an optional - but highly effective - strategy to enhance the overall **user experience**. Instead of forcing users to wait while the system performs **heavy tasks** like **embedding generation** or **similarity matching**, these operations are **offloaded** from the main application thread and handled **asynchronously**.

With this setup, the system can **instantly acknowledge** a product inquiry as soon as it's submitted, giving users the impression of **speed and responsiveness**. Meanwhile, all the **resource-intensive processing** continues quietly in the background. This design pattern **decouples user interaction** from backend logic, making the application feel significantly more responsive.

Typically, background workers are implemented using **task queues** like **Celery**, paired with a **message broker** such as **Redis** or **RabbitMQ**. When a new task - say, "**process this product inquiry**" - is triggered, it's added to the task queue. A separate **worker process** then picks it up and carries out the necessary computation independently.

The final results can be stored in a **database** or a **temporary cache**, ready to be retrieved once processing is complete. Users can be notified through various channels - such as **polling**, **email alerts**, or even a **real-time notification system** - depending on your preferred user flow.

This architecture is particularly valuable in applications where **high-latency operations** are unavoidable, but users still expect a **real-time interface**. By introducing a **background processing layer**, the system gains not just **efficiency**, but also **scalability** and **user satisfaction**.

6. Parallelize Image Downloads & Processing

When dealing with large volumes of **product images**, one of the most impactful optimizations we can make is to implement **parallelized image downloading and processing**. In the current approach, images are handled **sequentially** - meaning each image is downloaded and processed **one at a time**. This linear flow creates a major **bottleneck**, especially when working with **hundreds of images**, leading to extended wait times and inefficient use of system resources.

By adopting **parallelism**, you can unlock the full power of your system's **CPU cores** and **network bandwidth**, allowing **multiple images** to be downloaded and preprocessed **simultaneously**. This significantly **reduces total runtime**, often turning what would take **minutes** into just **seconds**, depending on the machine's capabilities.

This performance boost can be achieved using tools such as **Python's concurrent.futures.ThreadPoolExecutor** or **asyncio** for **asynchronous I/O operations**. With **ThreadPoolExecutor**, for instance, we can create a pool of threads - each responsible for downloading and preparing an image for model inference. These images can then be batched and fed into our model efficiently.

However, because **parallel processing** introduces **concurrency**, it comes with its own set of challenges. We will need to carefully manage **thread safety**, **exception handling**, and system constraints like **maximum thread count** and **network timeouts**. Despite the added complexity, this optimization is incredibly **effective** for systems where **throughput and speed** are critical.

For large-scale or production-grade deployments, **parallel image processing** can be a game-changer - dramatically enhancing performance while keeping users happy with faster turnaround times.

Rationale for Selecting Option 1: Precomputing and Storing Embeddings

When I precomputed the **text and image embeddings** for all existing products and stored them in the same **Azure Table**, the ml_model script was able to retrieve the stored embeddings directly instead of recalculating them during each inquiry. This simple yet effective change reduced the **runtime from 5 minutes to just 7 seconds**, resulting in a dramatic improvement in system performance and user experience.

This makes **Option 1: Precompute and Store Embeddings** the **best solution** in terms of **highest impact with minimal changes**. It required no major architectural overhauls, no additional services, and minimal code refactoring - just embedding once and saving the results. In contrast, **Option 2: Batch**

Inference, while powerful, involves reworking data pipelines and managing batch sizes dynamically. **Option 4: Using FAISS for similarity search** introduces complexity in vector indexing and requires setting up and maintaining a separate FAISS index. **Option 5: Background Workers (e.g., Celery)** adds system complexity by requiring task queues and async processing. **Option 6: Parallel Image Downloading/Processing** brings concurrency challenges and higher potential for timeouts or race conditions. Meanwhile, **Option 3: Switching to a lighter model (e.g., MobileNetV2)** may reduce runtime, but it **compromises accuracy**, which is not ideal for critical similarity matching. Therefore, **precomputing embeddings** offers the **most efficient and accurate optimization** with the **least development effort**.

Model Implementation

Newly added ***compute_image_embedding()*** method and other changes in the **product_upload** script allows to precompute text and image embeddings for each product using SBERT for text (*product_category and product_description*) and ResNet50 for images. It processes and encodes the embeddings into **base64 strings**, which are then stored in Azure Table Storage under the fields ***text_embedding_pc, text_embedding_pd,*** and ***image_embedding***.

```
36  # Models for embeddings
37  model_pc = SentenceTransformer('all-mpnet-base-v2')
38  model_pd = SentenceTransformer('bert-large-nli-stsb-mean-tokens')
39  model_img = ResNet50(weights='imagenet', include_top=False, pooling='avg')
40
41  def encode_array(arr):
42      return base64.b64encode(arr.astype(np.float32).tobytes()).decode('utf-8')
```

```
143 def compute_image_embedding(image_url):
144     try:
145         response = requests.get(image_url, timeout=10)
146         img = Image.open(BytesIO(response.content)).convert('RGB')
147         img = img.resize((224, 224))
148         img_array = image.img_to_array(img)
149         img_array = np.expand_dims(img_array, axis=0)
150         img_array = preprocess_input(img_array)
151         return model_img.predict(img_array)[0]
152     except Exception as e:
153         print(f"Error processing image {image_url}: {e}")
154         return np.zeros((2048,), dtype=np.float32)
```

```
160         # Compute embeddings
161         pc_embedding = model_pc.encode(row["product_category"])
162         pd_embedding = model_pd.encode(row["product_description"] if pd.notna(row["product_description"]) else "")
163         img_embedding = compute_image_embedding(row["image_url"]) if row["image_url"] else np.zeros((2048,), dtype=np.float32
```

```
179             # Encode and store embeddings
180             "text_embedding_pc": encode_array(pc_embedding),
181             "text_embedding_pd": encode_array(pd_embedding),
182             "image_embedding": encode_array(img_embedding)
```

Additionally, the **ml_model** script was also updated to leverage the precomputed embeddings by retrieving them directly during inference. This eliminated the need for on-the-fly computation, resulting in a significantly faster response time and a more efficient prediction process.

Repository Check-in Details

The scripts I have checked into the GitHub repository since Progress Report 3 are as follows:

- product_upload script
    - Modified to generate text and image embeddings for existing products during upload.
    - Modified to return only products that are included in the recycling program.
- ml_model script
    - Modified to align with the updated embedding structure in the product_upload script.
    - Modified to return the best matching product category using both text and image similarity.
    - Modified to measure the time taken to submit an inquiry.

Regular check-ins were maintained to ensure all progress was tracked in the repository.

References

- Hugging Face - SentenceTransformers Documentation: https://www.sbert.net/
- TensorFlow – Performance Guide for Serving Models: https://www.tensorflow.org/tfx/serving/performance
- MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications: https://arxiv.org/abs/1704.04861
- Facebook AI Research – FAISS GitHub: https://github.com/facebookresearch/faiss
- Celery Official Documentation: https://docs.celeryq.dev/en/stable/getting-started/introduction.html
- Python's *concurrent.futures* Documentation: https://docs.python.org/3/library/concurrent.futures.html
- ChatGPT: OpenAI